Prompt Library for the Complete Salesforce Certified Administrator with ChatGPT Course

By Mike Wheeler - Rapid Reskill

Using ChatGPT to Analyze the Company Settings Learning Objective

[Prompt]

The first learning objective in the Salesforce Certified Administrator Exam Guide is:

Describe the information found in the company settings (for example: company settings fiscal year, business hours, currency management, default settings).

Give me a conceptual prompt I should ask as a new Salesforce learner in order to fully understand these topics found in the first learning object: fiscal year, business hours, currency management and default settings, in Salesforce company settings.

Place these conceptual prompts in a table, with the prompts to the left and then respond to each prompt in the column on the right.

Using ChatGPT to Analyze the User Interface Settings Learning Objective

[Prompt]

The second learning objective in the Salesforce Certified Administrator Exam Guide is:

Distinguish and understand the administration of declarative configuration of the User Interface. (for example: UI settings, app menu, list views, global actions, Lightning App Builder)

Give me a conceptual prompt I should ask as a new Salesforce learner in order to fully understand these topics found in the learning objective: UI Settings, App Menu, List Views, Global Actions and Lightning App Builder.

Place these conceptual prompts in a table, with the prompts to the left and then respond to each prompt in the column on the right.

Profiles

[Prompt 1]

What is the difference between a Standard and a Custom Profile in Salesforce?

[Prompt 2]

What are limitations of Standard Profiles in Salesforce?

[Prompt 3]

You are an expert Salesforce administrator. Should users be assigned Standard Profiles or Custom Profiles? Explain to me why or why not?

Profiles vs. Permission Sets

[Prompt 1]

What is the main differences between profiles and permission sets in Salesforce and when should you use a permission set instead of a profile?

[Response]

In Salesforce, both profiles and permission sets are used to assign permissions and access levels to users. However, there are some key differences between the two.

A profile is a collection of settings and permissions that define what a user can and cannot do within an organization. It is assigned to a user and applies to all objects and records within the organization. Profiles are used to define the basic access levels and permissions for different types of users, such as sales reps, marketing users, and administrators.

On the other hand, a permission set is a collection of settings and permissions that can be assigned to individual users, in addition to their profile. Permission sets can be used to grant additional permissions to users who need them, without changing the permissions of their entire profile. This allows for more granular control over user access levels.

Here are some situations where you might want to use a permission set instead of a profile:

When you need to grant additional permissions to a specific user, but do not want to give those permissions to all users with the same profile.

When you have a user who needs to access certain objects or perform certain tasks that are not included in their profile.

When you want to grant temporary or one-time access to certain features or data.

When you need to grant different permissions to users who have the same profile.

Overall, profiles and permission sets are both important tools for managing user access and permissions in Salesforce, and understanding their differences is essential for effective user management.

Dissecting and Analyzing the Configuration and Setup Knowledge Area with ChatGPT

[Prompt 1]

You are a Salesforce expert and have been tasked with developing curriculum to help new Salesforce learners to pass the Salesforce Certified Administrator Exam. I will be giving you various learning objectives from the Administrator Exam Guide. Analyze each learning objective and write a 3-5 paragraph article on each topic presenting within each individual learning objective.

Here is the first learning objective:
Describe the information found in the company settings (for example: company settings fiscal year, business hours, currency management, default settings).

To help you identify topics in each learning objective, here are the topics in this first one:

Company Settings
Fiscal Year

Business Hours
Currency Management
Default Settings

Note that the Default Settings should be related to the default fields that can be set on the Company Information screen.

With subsequent prompts I will be giving you the learning objectives and will expect you to draw out the topics and generate the 3-5 paragraph articles for each.

[Response 1]

Learning Objective: Describe the information found in the company settings (for example: company settings fiscal year, business hours, currency management, default settings).

Salesforce provides a comprehensive set of tools for managing company settings to customize the system to suit a company's business needs. Company settings in Salesforce are essential as they contain information that is used across the platform, including information about the fiscal year, business hours, currency management, and default settings.

The fiscal year setting is used to track the financial performance of the organization. It is important to set this correctly, as it affects reporting and forecasting within the organization. To set the fiscal year, navigate to Setup, search for "Fiscal Year" in the Quick Find box, and then select "Fiscal Year Settings." Here, you can set the fiscal year start month, as well as the number of periods for the fiscal year.

Business hours in Salesforce help define the working hours of an organization. It is important to set this up accurately, as it affects the performance of service-level agreements (SLAs), escalation rules, and other time-dependent features within Salesforce. To set up business hours, navigate to Setup, search for "Business Hours" in the Quick Find box, and then select "Business Hours." Here, you can define business hours for each day of the week.

Currency management in Salesforce allows companies to manage the currencies they use for their business transactions. Salesforce supports over 150 currencies, allowing companies to work with customers and suppliers worldwide. To set up currency management, navigate to Setup, search for "Currency" in the Quick Find box, and then select "Currency Management." Here, you can set up new currencies, activate/deactivate existing currencies, and set default currencies.

Default settings in Salesforce allow companies to customize their organization's information, such as the company name, address, and logo. To access these settings, navigate to Setup, search for "Company Information" in the Quick Find box, and then select "Company

Information." Here, you can update the default fields for your organization, such as the default time zone, default locale, and default language.

In summary, the company settings in Salesforce are an essential part of the platform. Accurately setting the fiscal year, business hours, currency management, and default settings is essential for organizations to get the most out of their Salesforce implementation. It is important to note that these settings can have a significant impact on the performance of other features within the platform, such as reporting, forecasting, and SLAs.

[Prompt 2]

Expound upon the Business Hours paragraph and give me 4 key points to keep in mind. These can include somewhat related topics such as Holidays, Entitlements and whatever else may be important for a new learner to understand to bring the Business hours concept to life. Also, give a scenario where Business Hours would come into use in a real world setting.

[Response]

Business hours in Salesforce are a critical component of the platform and define the working hours of an organization. To help new learners understand this concept better, here are four key points to keep in mind:

Business hours affect time-dependent features: Time-dependent features, such as escalation rules and service-level agreements (SLAs), are based on business hours. It is important to ensure that business hours are set up accurately, as this can affect the performance of these features.

Holidays and vacation days: Salesforce allows organizations to set up holidays and vacation days within their business hours to reflect their working schedule. By defining these days, companies can ensure that their service levels are met and that their customers receive timely responses.

NOTE FROM MIKE: Vacation days is not a standard functionality out-of-the-box with Salesforce. You would need to build a custom solution to accommodate vacation days.

Entitlements: Entitlements in Salesforce are based on business hours and define the amount of support that customers are entitled to. By setting up business hours accurately, companies can ensure that entitlements are calculated correctly and that customers receive the level of support they are entitled to.

Real-time data: Salesforce provides real-time data about the status of cases, leads, and other objects within the platform. By setting up business hours accurately, companies can ensure that this data is accurate and up-to-date.

Scenario: Suppose a company operates in multiple time zones and has a support team that operates in different regions. The support team is responsible for resolving customer issues and has a service-level agreement (SLA) of 24 hours. The company's business hours are set up to reflect the working hours of the support team in each region, and holidays and vacation days are defined accordingly. If a customer submits a support ticket outside of business hours, Salesforce will not start the SLA timer until the next business day, ensuring that the SLA is met. By accurately setting up business hours and holidays, the company can provide excellent customer support and meet its SLAs.

[Prompt 3]

Here is the final learning objective:

Given a scenario, determine the appropriate use of a custom profile or permission set using the various profile settings and permissions.

NOTE FROM MIKE: You can repeat this process for the other learning objectives in the Exam Guide.

[Response]

Learning Objective: Given a scenario, determine the appropriate use of a custom profile or permission set using the various profile settings and permissions.

In Salesforce, profiles and permission sets are used to control access to data and features within the platform. A profile is a collection of settings and permissions that define what a user can see and do in Salesforce, while a permission set is a collection of settings and permissions that can be assigned to users to grant additional access to specific features or data. To help new learners understand this concept better, let's explore how to determine the appropriate use of a custom profile or permission set using various profile settings and permissions.

The first step in determining the appropriate use of a custom profile or permission set is to understand the specific requirements of the scenario. This could include understanding which users require access to specific features or data, and which users should be restricted from accessing them. Once the requirements have been established, the next step is to identify the relevant profile settings and permissions that will be needed to meet those requirements.

Some of the key profile settings that can be used to control access to data and features in Salesforce include:

Object settings: Object settings control access to standard and custom objects in Salesforce. They define whether a user can view, create, edit, or delete records, as well as whether they can see specific fields on those records.

Tab settings: Tab settings control which tabs are visible to users in the Salesforce user interface. They can be used to hide tabs that are not relevant to a user's job function or restrict access to sensitive data.

Apex class and Visualforce page access: Apex class and Visualforce page access can be controlled using profile settings. This is useful when you want to restrict access to custom functionality or data that is only accessible via Apex or Visualforce.

Record types and page layouts: Record types and page layouts can be used to control the layout and visibility of data on a record-by-record basis. This is useful when you want to customize the user experience for specific users or groups of users.

When it comes to permission sets, the key advantage is that they can be assigned to users in addition to their profile, allowing them to access additional features or data that are not available to their profile. Some examples of features that can be enabled using permission sets include:

Campaign management: Permission sets can be used to enable access to campaign management features, such as creating campaigns, adding members to campaigns, and tracking campaign metrics.

Lightning Experience: Permission sets can be used to enable access to the Lightning Experience user interface, which provides a more modern and intuitive user experience compared to Salesforce Classic.

Reports and dashboards: Permission sets can be used to enable access to specific reports and dashboards, allowing users to analyze data in new ways. - NOTE FROM MIKE: While you can adjust the security and access to reports and dashboard folders by way of permission sets, you can't enable access to specific reports and dashboards by way of a permission set. As always with ChatGPT, you will want to verify the accuracy of generated information by double-checking actual functionality in the Salesforce interface, as well as Salesforce's online help. You can also debate the accuracy of ChatGPT's responses with ChatGPT and say things along the lines of 'that seems incorrect' or 'I believe that is not accurate' to see if it can realize its mistake or provide further context.

Custom objects and fields: Permission sets can be used to enable access to custom objects and fields that are not available to a user's profile.

In summary, determining the appropriate use of a custom profile or permission set requires an understanding of the specific requirements of the scenario, as well as the relevant profile settings and permissions that can be used to meet those requirements. By using profiles and permission sets effectively, organizations can control access to data and features within Salesforce, ensuring that users have the access they need to perform their jobs effectively while maintaining the security of their data.

Dissecting and Analyzing Trailhead Trailmixes with ChatGPT

[Prompt 1]

Please analyze the first knowledge area of the Salesforce Certified Administrator Trailmix. Total up the number of hours and give me a schedule to where I can complete this knowledge area of the Trailmix in one week's time. I want to take off Saturday's.

Provide me with a daily study schedule, noting what needs to be completed each day. And also give me project ideas that I can perform in addition to this Trailmix to help me get more hands on experience, perhaps apps I could create as well.

[Prompt 2]

Give me 5 app ideas that I could build. Keep these fairly basic for new salesforce admins.

[Prompt 3]

What objects would need to be created for app number 1. Also, what would I need to configure in my org, users and various security settings in order to get more hands on experience in those areas?

[Prompt 4]

Let's not create a custom object for contacts but instead use the standard contact object in salesforce. Come up with a fictional scenario for this app where we are tracking our contacts that are customers and those that have bought a product from our company. Come up with a fictional product that can be our custom object and tell me how to create that custom object.

Give me any further details needed for the data model, such as fields I would need to create on the contact and/or custom object and anything else that may prove helpful.

Dissecting and Analyzing Trailhead Trails with ChatGPT

[Prompt 1]

Provide a one-paragraph overview of all of these key topics from the first knowledge area of salesforce certified administrator exam:

UI features
Company information
Locale settings
Search results
List views
Currency management
Fiscal year
Default settings
Homepage layouts
User management
Security settings
Sharing settings

[Prompt 2]

That is not much detail. Give an individual paragraph of detail and explanation for each of these topics:

UI features
Company information
Locale settings
Search results
List views
Currency management
Fiscal year
Default settings
Homepage layouts
User management

Security settings
Sharing settings

Creating Configuration and Setup Practice Test Questions with ChatGPT

Please answer this Salesforce Administrator Practice Test Question:

Sales representatives at Ursa Major Solar are working on opportunities and need to see how their colleagues have effectively managed other opportunities with comparable products, competing against the same competitors.

Which two features should an administrator use to allow for this?

Choose 2 options.

A
Opportunity update reminders
B
Opportunity Dashboard
C
Chatter groups
D
Big deal alerts

Using ChatGPT to Analyze Object Manager and Lightning App Builder Learning Objectives

[Prompt 1]

Describe the standard object architecture and relationship model of Salesforce.

[Response]

Salesforce follows a standard object architecture and relationship model, which provides a consistent and unified way of organizing and storing data within the platform. The standard object architecture consists of a set of pre-defined objects that are available out-of-the-box,

such as accounts, contacts, leads, opportunities, cases, and more. These objects represent different types of data that are commonly used in sales, marketing, and customer service processes.

Each object in Salesforce has a set of fields, which are used to store specific pieces of information about the object. For example, the Account object might have fields like Account Name, Industry, Annual Revenue, and Billing Address. The Contact object might have fields like First Name, Last Name, Email, Phone, and Mailing Address.

In addition to the standard objects, Salesforce also provides the ability to create custom objects, which can be used to store data that is unique to a particular organization or business process. Custom objects are created using a point-and-click interface, which makes it easy for users to define the fields and relationships that are needed.

The relationship model in Salesforce is based on the concept of related objects. Each object can be related to one or more other objects, creating a hierarchy of related data. For example, an Account object might be related to one or more Contact objects, representing the people who work at the account. A Contact object might be related to one or more Opportunity objects, representing the potential sales opportunities that are associated with the contact.

Relationships between objects are defined using lookup fields or master-detail relationships. A lookup field allows an object to reference another object, while a master-detail relationship creates a parent-child relationship between objects, with the child object dependent on the parent object.

Overall, the standard object architecture and relationship model in Salesforce provides a flexible and scalable way to organize and manage data within the platform.

[Prompt 2]

Please tell me more about object oriented principles.

[Prompt 3]

tell me more about objects

[Prompt 4]

What are the implications of deleting fields in Salesforce?

Using ChatGPT to Generate Custom Object Ideas

[Prompt]

You are an expert Salesforce Consultant and have build many applications and solutions on the platform for Marketing, Sales and Service. I want to learn more about Salesforce, specifically related to the creation of Custom objects. Give me 10 different custom object examples I could create inside of Salesforce, and what those custom objects may be used for. Also provide me with a list of fields and relationships I may need to establish on those custom objects.

[Response]

Sure, here are 10 different custom object examples that you could create inside of Salesforce, along with their potential uses and recommended fields and relationships:

Project Management Object: A custom object to manage and track projects within Salesforce.
Fields: Project Name, Description, Start Date, End Date, Project Manager, Project Status, Budget, Tasks.

Relationships: Many-to-One with User object, One-to-Many with Task object.

Asset Management Object: A custom object to manage and track physical assets.
Fields: Asset Name, Description, Location, Purchase Date, Manufacturer, Model, Serial Number.

Relationships: One-to-Many with Contact object, Many-to-One with Account object.

Warranty Management Object: A custom object to manage and track warranties associated with assets.
Fields: Warranty Name, Description, Warranty Provider, Start Date, End Date, Warranty Status, Asset.

Relationships: Many-to-One with Asset object.

Event Management Object: A custom object to manage and track events within Salesforce.
Fields: Event Name, Description, Start Date, End Date, Location, Organizer, Attendees.

Relationships: Many-to-One with User object.

Issue Tracking Object: A custom object to manage and track issues reported by customers.
Fields: Issue Description, Severity, Status, Customer, Contact, Account.

Relationships: Many-to-One with Contact object, Many-to-One with Account object.

Product Management Object: A custom object to manage and track products within Salesforce.
Fields: Product Name, Description, Manufacturer, Model, Price, Product Status.

Relationships: Many-to-One with Account object.

Contract Management Object: A custom object to manage and track contracts within Salesforce.
Fields: Contract Name, Description, Start Date, End Date, Contract Type, Customer, Contact, Account.

Relationships: Many-to-One with Contact object, Many-to-One with Account object.

Survey Object: A custom object to manage and track survey responses within Salesforce.
Fields: Survey Name, Description, Questions, Responses.

Relationships: Many-to-One with User object.

Certification Management Object: A custom object to manage and track employee certifications.
Fields: Certification Name, Description, Certification Date, Expiration Date, Employee.

Relationships: Many-to-One with User object.

Recruiting Object: A custom object to manage and track job openings and candidates.
Fields: Job Title, Job Description, Hiring Manager, Candidate Name, Candidate Contact Information, Resume.

Relationships: Many-to-One with User object, Many-to-One with Contact object.

Keep in mind that these are just examples and you can customize them as per your business needs. I hope this helps you get started with creating custom objects in Salesforce!

Creating Additional Custom Fields

[Prompt]

For the Warranty Management Object, tell me more about the Warranty Provider field. Could that Warranty Provider field be related to the Account object? If so, should I make that a Lookup Relationship or a Master-Detail Relationship?

[Response]

Yes, the Warranty Provider field in the Warranty Management Object could be related to the Account object in Salesforce. Since there can be multiple warranties associated with a single account, it would be appropriate to create a Lookup Relationship between the Warranty Management Object and the Account object.

A Lookup Relationship creates a simple link between two objects where the Warranty Management Object has a field that references an Account record. This relationship allows you to create a Warranty record and associate it with an existing Account record.

In contrast, a Master-Detail Relationship is typically used when there is a parent-child relationship between two objects and the child object record cannot exist without a parent object record. However, in the case of the Warranty Management Object and Account object, warranties can exist without the need for an associated account, so a Lookup Relationship would be more appropriate.

Creating Picklist Fields

[Prompt]

For the Warranty Status field, should that be a picklist field? And if so, what are 4 or 5 good example status designations that I should include as options?

[Response]

Yes, the Warranty Status field could be a picklist field in the Warranty Management Object. This would allow you to standardize the values that are entered for this field and make it easier to report on and analyze the warranty data.

Here are some examples of Warranty Status designations that you could include as options in the picklist field:

Active - The warranty is currently in effect and has not yet expired.
Expired - The warranty has expired and is no longer in effect.
Pending - The warranty is pending approval or verification.
Void - The warranty has been voided due to a violation of the terms and conditions.
Cancelled - The warranty has been cancelled by the warranty provider or customer.
Of course, you can customize the list of options based on your specific business needs and requirements.

Converting a Lookup Field to a Master-Detail Relationship

[Prompt]

Give me examples of master/detail fields I could create on the warranty object.

[Response]

Sure, here are some examples of Master-Detail fields that you could create on the Warranty object:

Warranty Line Item - A Master-Detail field that links the Warranty object to a Warranty Line Item object, which contains information about the specific items covered under the warranty, such as product name, quantity, and serial number.

Warranty Claim - A Master-Detail field that links the Warranty object to a Warranty Claim object, which contains information about claims made against the warranty, such as claim date, claim description, and resolution status.

Service Appointment - A Master-Detail field that links the Warranty object to a Service Appointment object, which contains information about scheduled appointments related to the warranty, such as appointment date, location, and service type.

Repair Estimate - A Master-Detail field that links the Warranty object to a Repair Estimate object, which contains information about the estimated cost and time required for repairs covered under the warranty.

Warranty Contact - A Master-Detail field that links the Warranty object to a Warranty Contact object, which contains information about the primary point of contact for the warranty, such as name, phone number, and email address.

Keep in mind that these are just examples and the specific Master-Detail fields you create will depend on your business requirements and how you want to track and manage your warranty data.

Using ChatGPT for Junction Object Ideas

[Prompt]

What is a Junction Object in Salesforce?

[Response]

In Salesforce, a Junction Object is a custom object that is used to create a many-to-many relationship between two other custom or standard objects.

Junction objects are used when you need to create a relationship between two objects that don't have a direct relationship with each other. For example, if you have a custom object for Contacts and a custom object for Events, you may want to track which Contacts are attending which Events. You can create a junction object called "Attendees" that has Master-Detail relationships to both the Contacts and Events objects. This allows you to create a many-to-many relationship between Contacts and Events, and track which Contacts are attending which Events.

Junction objects typically have two Master-Detail relationships, one to each of the objects it is connecting. The fields on the junction object represent the relationship between the two objects. For example, in the Attendees junction object, you might have fields for the Contact Name, Event Name, and Attendee Type.

Junction objects are a powerful feature in Salesforce and are commonly used to track complex relationships between objects.

[Prompt 2]

What are some junction objects that we could create for the warranty management functionality?

[Response]

Here are a few examples of Junction Objects that you could create for Warranty Management functionality:

Warranty Coverage - A junction object that links the Warranty object with the Product object, allowing you to track which products are covered by which warranties. This could be useful if you have different types of warranties for different products.

Warranty Claim Line Item - A junction object that links the Warranty Claim object with the Warranty Line Item object, allowing you to track which specific items were included in each warranty claim. This could be useful if you want to track the types of products or services that are most commonly claimed under warranty.

Warranty Contact Role - A junction object that links the Warranty object with the Contact object, allowing you to track the different roles that contacts can have in relation to the warranty. For example, you might have contacts who are authorized to make warranty claims, contacts who are responsible for approving claims, and contacts who are designated as warranty administrators.

Warranty Repair - A junction object that links the Warranty Claim object with the Repair Estimate object, allowing you to track the estimated and actual costs associated with each warranty repair. This could be useful for tracking repair costs over time and identifying areas where improvements can be made.

Keep in mind that these are just examples, and the specific junction objects you create will depend on your business requirements and the relationships between the different objects you are tracking.

Assignment: Creating a Junction Object

[Prompt]

I have a custom object called Warranty created in my Salesforce instance. That Warranty object has a master-detail field to the Contact object that I have labeled as Warranty Contact. This was intended as the Primary contact for the warranty. I want to introduce warranty contact role as signified as your option #3 above. What would I need to create in order to have a junction object between contact and warranty in order to establish that junction relationship between contact and warranty? Give me the name of the junction object and whatever fields I would need to create, especially the master-detail ones.

[Response]

To create a junction object between Contact and Warranty to establish the Warranty Contact Role relationship, you could create a new custom object called "Warranty Contact Role" with the following fields:

Warranty (Master-Detail) - This field links to the Warranty object and establishes a Master-Detail relationship. This field would be used to indicate which Warranty the Warranty Contact Role is associated with.

Contact (Master-Detail) - This field links to the Contact object and establishes a Master-Detail relationship. This field would be used to indicate which Contact is associated with the Warranty Contact Role.

Role (Picklist or Text) - This field would be used to indicate the specific role that the contact plays in relation to the warranty. For example, "Authorized Claimant", "Warranty Administrator", "Approver", etc.

Start Date (Date) - This field would be used to indicate the start date of the contact's role in relation to the warranty.

End Date (Date) - This field would be used to indicate the end date of the contact's role in relation to the warranty.

By creating this junction object, you can establish the Warranty Contact Role relationship between the Warranty and Contact objects, allowing you to track the specific role that each contact plays in relation to each warranty. The junction object would be named "Warranty Contact Role" and would have Master-Detail relationships to both the Warranty and Contact objects.

[Prompt 2]

I am using a picklist for role. Please give me 10 additional options to add to the 3 that you gave me previously.

Deleting Fields and Implications

[Prompt]

What are the implications of deleting fields in Salesforce?

[Response]

Deleting fields in Salesforce can have various implications, depending on the type of field and the context in which it is used. Here are some potential implications to consider:

Data loss: Deleting a field will permanently remove all data stored in that field for all records. Before deleting a field, you should ensure that you have a backup of any data that you need to keep.

Impact on functionality: Deleting a field can impact any functionality that relies on that field. For example, if you delete a field that is used in a report or a formula field, those components may no longer function correctly.

Impact on validation rules and workflow rules: Deleting a field can impact any validation rules or workflow rules that use that field. If a validation or workflow rule references a deleted field, it will not function correctly.

Impact on page layouts and record types: If you delete a field that is used in a page layout or a record type, those components may not function correctly. For example, if you delete a required field that is used in a page layout, users will not be able to save records that use that layout.

Impact on integrations: Deleting a field can impact any integrations that use that field. If an external system relies on a field that you delete, the integration may fail or not function correctly.

In general, it is recommended to avoid deleting fields whenever possible, and instead consider alternatives such as hiding the field from page layouts or deactivating the field if it is no longer needed. If you do need to delete a field, it is important to carefully consider the potential impacts and to take steps to mitigate those impacts before deleting the field.

Using ChatGPT to Create Formula Fields

[Prompt]

Give me 10 different formulas that could be created in Salesforce. Provide these in a table with the scenario on the left, the return type for the formula in a middle column, and the formula code on the right.

Scenario	Return Type	Formula Code
Calculate the age of a contact	Number	YEAR(TODAY()) - YEAR(Contact.Birthdate__c)

Calculate the total amount of a deal	Currency	Opportunity.Amount
Calculate the number of days between two dates	Number	(Date2__c - Date1__c)
Concatenate first name and last name	Text	Contact.FirstName & " " & Contact.LastName
Calculate the total number of closed won opportunities for an account	Number	SUM(IF(Opportunity.AccountId = Account.Id && Opportunity.StageName = 'Closed Won', 1, 0))
Calculate the commission amount for a salesperson based on their total sales	Currency	IF(Salesperson.Total_Sales__c > 100000, Salesperson.Total_Sales__c * 0.1, Salesperson.Total_Sales__c * 0.05)
Calculate the days since a lead was last contacted	Number	TODAY() - Lead.Last_Activity_Date
Determine if an account is a high-value account based on their total annual revenue	Checkbox	IF(Account.Annual_Revenue__c > 1000000, true, false)
Calculate the number of days until a project deadline	Number	Deadline__c - TODAY()
Calculate the percent of tasks completed for a project	Percent	(SUM(Project.Tasks_Completed__c) / SUM(Project.Total_Tasks__c)) * 100

Using ChatGPT for Mass Assigning Account Record Types

[Prompt]

I have just introduced Record Types on the Account object in my Salesforce instance. I need to update all of my existing account records that currently do not have a Record Type assigned to now be assigned the account record type of Prospect. How can I do this?

Creating Rollup Summary Fields

[Prompt]

Can you create a rollup summary field on an account in salesforce to give you the total dollar amount of all closed won opportunities for an account?

Mass Updating Accounts Using a Rollup Summary Field

[Prompt]

That worked great. I have now updated all of those accounts to prospect. Since then, I have also introduced a rollup summary field on account called Total_Closed_Won_Opportunities__c which sums the total of all closed won opportunities for an account. I need to perform a mass update on all of my accounts in Salesforce so that any accounts with a record type of prospect and a Total_Closed_Won_Opportunities__c value of greater than 0 will be changed to a Client account record type. In my instance, the prospect account record type id is '' and the client account record type is ''.

[Reponse]

To update all Account records with a Prospect Record Type and a Total_Closed_Won_Opportunities__c value greater than 0 to a Client Record Type, you can use the following code:

```
List<Account> accountsToUpdate = [SELECT Id, RecordTypeId,
Total_Closed_Won_Opportunities__c FROM Account WHERE RecordTypeId =
'012Dp0000000BioIAE' AND Total_Closed_Won_Opportunities__c > 0];
for(Account acc : accountsToUpdate){
   acc.RecordTypeId = '012Dp0000000BieIAE';
}
update accountsToUpdate;
```

This code will retrieve all Account records with a Prospect Record Type and a Total_Closed_Won_Opportunities__c value greater than 0, and update them to the Client Record Type.
Note that you may need to adjust the Record Type Ids in the code to match your specific instance. Also, as with any mass updates, it is recommended to test this code on a smaller subset of data before running it on all records.

Analyzing the Object Manager and Lightning App Builder Knowledge Area with ChatGPT

[Prompt]

You are a Salesforce expert and have been tasked with developing curriculum to help new Salesforce learners to pass the Salesforce Certified Administrator Exam. I will be giving you various learning objectives from the Administrator Exam Guide. Analyze each learning objective and write a 3-5 paragraph article on each topic presented within each individual learning objective.

Here are the learning objectives for this knowledge area that I would like for you to analyze and dissect for me:

Describe the standard object architecture and relationship model. (for example: standard object, parent/child, master detail/lookup/junction relationships, and record types.)
Explain how to create, delete, and customize fields and page layouts on standard and custom objects, and know the implications of deleting fields.
Given a scenario, determine how to create and assign page layouts, record types and business processes for custom and standard objects.

[Response]

Learning Objective: Describe the standard object architecture and relationship model.

Salesforce's standard object architecture is the framework that defines how objects, records, and relationships are organized within the platform. The standard objects are pre-built objects that come with the Salesforce platform, such as Accounts, Contacts, Opportunities, and Cases, to name a few. The object architecture of standard objects is organized into a parent-child hierarchy, where the parent object has one or more child objects associated with it. For example, the Account object is the parent of the Contact object, as each Contact record is associated with a single Account record.

Additionally, the standard object architecture includes several types of relationships between objects. Master-detail relationships represent a parent-child relationship in which the child record inherits certain characteristics, such as field-level security, from the parent record. Lookup relationships, on the other hand, represent a weaker relationship between two objects, and allow a record to be associated with a record of another object. Lastly, junction relationships enable many-to-many relationships between two objects.

Finally, the standard object architecture also includes the concept of record types, which allows organizations to configure different sets of fields, picklist values, and page layouts for a single object. Record types can be used to support different business processes and can be customized for different profiles and user groups.

Learning Objective: Explain how to create, delete, and customize fields and page layouts on standard and custom objects, and know the implications of deleting fields.

Creating, deleting, and customizing fields and page layouts on standard and custom objects is an essential part of configuring Salesforce for an organization's unique business needs. Custom fields can be created to store data that is not already captured in standard fields, and can be used to create new reports, dashboards, and automation rules. Similarly, page layouts can be customized to display only the relevant information to users and can be assigned to different profiles and record types.

However, it is important to understand the implications of deleting fields or page layouts. When a field is deleted, all the data stored in that field is also deleted permanently. Additionally, if the field is referenced in other objects, such as in a report or workflow rule, the deletion of the field will cause those references to break, resulting in errors. Therefore, it is important to consider the impact of field deletion before proceeding.

Similarly, deleting a page layout can result in users losing access to important data, as fields and related lists may no longer be visible. Instead of deleting page layouts, it is recommended to assign new page layouts to users or profiles.

Learning Objective: Given a scenario, determine how to create and assign page layouts, record types, and business processes for custom and standard objects.

Creating and assigning page layouts, record types, and business processes for custom and standard objects can be a complex task, but it is essential to ensure that users have access to the relevant data and tools they need to complete their work. To create a new page layout, a Salesforce admin must first create a new page layout template and then customize it by dragging and dropping fields and related lists onto the layout.

Similarly, creating a new record type requires defining the fields, picklist values, and page layout that will be available for the new record type. Once the new record type is created, it can be assigned to specific profiles or users.

Lastly, business processes can be created to automate the workflow for an object. This includes defining the stages that a record can go through, the fields that are required or optional at each stage, and the actions that can be taken when a record moves from one stage to another.

In conclusion, mastering these Salesforce Certified Administrator Exam learning objectives requires understanding the standard object architecture and relationship model, creating, deleting, and customizing fields and page layouts on standard and custom objects, and being able to create and assign page layouts, record types, and business processes for custom and standard objects. By having a strong understanding of these concepts, Salesforce admins can configure Salesforce to fit their organization's unique needs, enabling their users to work more efficiently and effectively.

It is important to note that while these are separate learning objectives, they are all interconnected. For example, creating a new record type requires customizing the fields and page layout, and may also require updating business processes to support the new record type. Similarly, customizing page layouts may involve creating new fields or updating existing fields.

Overall, by taking a holistic approach to Salesforce customization, admins can ensure that their organization is making the most of the platform's capabilities, and users are able to work more efficiently and effectively.

Creating an Object Manager and Lightning App Builder Study Guide with ChatGPT

[Prompt]

Take the information you have just provided me and create a Study Guide outline based on that material. Provide an overview section as well as a hands-on section. Come up with an app that I could build to apply the principles in the Study Guide so as to fully learn and comprehend the Object Manager and Lightning App Builder learning objectives. Also be sure to go into the

differences between the Lightning App Builder and the Lightning App Manager and include custom lightning home pages as well as the Home tab for a navigation item in a lightning app. Give an overview paragraph for each section as well.

[Response]

Study Guide Outline: Object Manager and Lightning App Builder

Overview:
The Object Manager and Lightning App Builder are powerful tools that enable Salesforce administrators to customize and configure Salesforce to fit their organization's unique needs. The Object Manager allows admins to create, modify, and manage objects and their related components, while the Lightning App Builder enables admins to create custom Lightning pages and applications.

Hands-On:
To fully learn and comprehend the Object Manager and Lightning App Builder learning objectives, it is recommended to build a sample app that incorporates these principles. The app should include standard and custom objects, along with custom fields, page layouts, record types, and business processes. The app should also include custom Lightning pages and applications, utilizing the Lightning App Builder.

App:
The sample app should be a simple project management app that tracks tasks and projects. It should include a custom object for tasks, with fields for task name, description, due date, status, and assigned user. It should also include a custom object for projects, with fields for project name, description, start date, end date, and project manager.

The app should utilize the Object Manager to create and manage these objects, along with their related components such as page layouts, record types, and business processes. The app should also utilize the Lightning App Builder to create custom Lightning pages and applications for the app.

Overview: Differences between Lightning App Builder and Lightning App Manager
It is important to understand the differences between the Lightning App Builder and the Lightning App Manager. The Lightning App Builder is a tool that enables admins to create custom Lightning pages and applications. The Lightning App Manager, on the other hand, is used to manage and customize Salesforce apps that are built using the Lightning Experience.

Overview: Custom Lightning Home Pages
Custom Lightning Home Pages are a powerful way to personalize the user experience in Salesforce. These pages can be customized to display relevant information and quick links, making it easier for users to access the information they need. Custom Lightning Home Pages can be created using the Lightning App Builder.

Overview: Home Tab as a Navigation Item in a Lightning App
The Home tab is the default landing page for Salesforce users. In a Lightning app, the Home tab can be customized to include components that provide quick access to frequently used records or actions. This can be done using the Lightning App Builder, and can help increase user efficiency by reducing the time it takes to access important information.

[prompt 2]
Give me three to five steps for a sample business process that I could build for this sample app. Also, can we use the standard Task object rather than creating a custom one?

[response]
Certainly! Using the standard Task object is a great option, as it already includes many of the fields and functionality that we would need for a basic project management app. Here are three steps for a sample business process that we could build for this app:

Task Creation: The first step in our business process would be to create a new task whenever a new project is added to the system. This could be accomplished by creating an automation in Process Builder that triggers whenever a new project is created. The automation would create a new task with the project name as the task subject, and assign it to the project manager.

Task Assignment: Once a task has been created, it would need to be assigned to the appropriate user. This could be accomplished by creating a validation rule that checks to ensure that the task is assigned to a user before it can be marked as completed.

Task Completion: Finally, we would need to ensure that tasks are completed in a timely manner. This could be accomplished by creating a custom field on the task object to track the task's due date. We could then create an automation in Process Builder that sends an email reminder to the assigned user when a task is overdue.

Of course, these are just a few sample steps for a business process, and the specific needs of your organization may vary. The important thing is to take a holistic approach to your business process design, and ensure that all of your automation and customizations are working together to support your overall business goals.

Leads Creation Overview

[Prompt]

What is a Lead in Salesforce?

[Response]

In Salesforce, a Lead is an individual or an organization that has shown some level of interest in a company's product or service but has not yet become a customer. Leads can be generated from various sources such as marketing campaigns, trade shows, website registrations, and more.

When a Lead is created in Salesforce, it typically includes basic contact information such as name, phone number, email address, and company name. Additional details such as the Lead's job title, company size, and industry may also be included.

Sales reps can then use the Lead record to track their interactions with the Lead, including phone calls, emails, and meetings. They can also qualify the Lead by assessing their level of interest and determining whether they meet the criteria to become a Sales Opportunity.

Once a Lead is qualified, it can be converted into an Account, Contact, and Opportunity record in Salesforce, allowing sales reps to continue tracking and managing the customer relationship as it progresses through the sales pipeline.

Using ChatGPT to Format Our Web-to-Lead Form

[Prompt]

Please provide more modernized styling to this web-to-lead code that was generated by Salesforce:

(paste your own web to lead form code here)

[Prompt 2]

Please update the code to include a more modern and responsive style and place that updated code inside of a code box.

Setting Lead Source for Web-to-Lead Forms

[Prompt]

Please update the code to add a hidden line for Lead Source and set that to Web.

[Prompt 2]

Please improve the styling and update the code so that the form has a more modern look. Use bootstrap.

[Response]

```
<!-- Add this meta tag to your page head -->
<meta http-equiv="Content-type" content="text/html; charset=UTF-8">

<!-- Add the Bootstrap CSS and JS files to your page head -->
<link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.0/css/bootstrap.min.css">
<script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
<script
src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.9.3/dist/umd/popper.min.js"></script>
<script src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.0/js/bootstrap.min.js"></script>

<!-- Add this form to your page body -->
<form action="https://webto.salesforce.com/servlet/servlet.WebToLead?encoding=UTF-8"
method="POST" class="needs-validation" novalidate>

  <input type="hidden" name="oid" value="">
<!--replace with your org id which can be found in company settings →
  <input type="hidden" name="retURL" value="https://mikewheelermedia.com">
  <input type="hidden" name="lead_source" value="Web">

  <div class="form-group">
    <label for="first_name">First Name</label>
    <input id="first_name" type="text" name="first_name" class="form-control" maxlength="40"
required>
    <div class="invalid-feedback">
      Please enter your first name.
    </div>
  </div>

  <div class="form-group">
```

```
    <label for="last_name">Last Name</label>
    <input id="last_name" type="text" name="last_name" class="form-control" maxlength="80"
required>
    <div class="invalid-feedback">
      Please enter your last name.
    </div>
  </div>

  <div class="form-group">
    <label for="email">Email</label>
    <input id="email" type="email" name="email" class="form-control" maxlength="80" required>
    <div class="invalid-feedback">
      Please enter a valid email address.
    </div>
  </div>

  <div class="form-group">
    <label for="company">Company</label>
    <input id="company" type="text" name="company" class="form-control" maxlength="40">
  </div>

  <button type="submit" class="btn btn-primary">Submit</button>

</form>

<!-- Add this script to your page body to handle form validation -->
<script>
  // Enable form validation using Bootstrap
  (function() {
    'use strict';

    window.addEventListener('load', function() {
      var form = document.getElementsByClassName('needs-validation')[0];
      form.addEventListener('submit', function(event) {
        if (form.checkValidity() === false) {
          event.preventDefault();
          event.stopPropagation();
        }
        form.classList.add('was-validated');
      }, false);
    }, false);
  })();
</script>
```

Mass Updating Legacy Lead Records without Record Types

[Prompt]

How can I query my Salesforce instance to get a count of all leads in my org?

[Response]

You can query your Salesforce instance using Salesforce Object Query Language (SOQL) to get the count of all leads in your org.

Here's an example query:

SELECT COUNT() FROM Lead

This query selects the Lead object and returns a count of all records in that object, which represents the count of all leads in your org. You can run this query using the Salesforce Developer Console or through an API tool such as Workbench or Postman.

[Prompt 2]

Give me the updated code to only query open leads instead of all.

[Response]

Sure! To query only open leads, you can add a WHERE clause to your SOQL query that filters for the `Status` field being equal to `Open`. Here's an example query:

SELECT COUNT() FROM Lead WHERE Status = 'Open'

This query selects the Lead object, filters for records where the `Status` field is equal to `Open`, and returns a count of those records, which represents the count of open leads in your org.

You can run this query using the Salesforce Developer Console or through an API tool such as Workbench or Postman.

[Prompt 3]

Update the code to instead exclude Closed - Converted and Closed - Not Converted status designations.

[Response]

To exclude the `Closed - Converted` and `Closed - Not Converted` status designations, you can modify the `WHERE` clause of your SOQL query to filter for `Status` not equal to those values. Here's an example query:

SELECT COUNT() FROM Lead WHERE Status NOT IN ('Closed - Converted', 'Closed - Not Converted')

This query selects the Lead object, filters for records where the `Status` field is not equal to `Closed - Converted` or `Closed - Not Converted`, and returns a count of those records, which represents the count of open leads in your org, excluding the leads that have been closed.

You can run this query using the Salesforce Developer Console or through an API tool such as Workbench or Postman.

[Prompt 4]

Give me the code to execute anonymous to update all Leads without a Record Type and update them to the Generator lead record type for all Lead Records that have either GC5000 series, GC3000 series or GC1000 series selected in their Product Interest field.

[Response]

Here's an example code snippet that you can use to update all Leads without a Record Type and update them to the Generator lead record type for all Lead Records that have either GC5000 series, GC3000 series or GC1000 series selected in their Product Interest field:

```
List<Lead> leadsToUpdate = new List<Lead>();
for (Lead lead : [SELECT Id, RecordTypeId, Product_Interest__c FROM Lead WHERE RecordTypeId = null AND Product_Interest__c IN ('GC5000 series', 'GC3000 series', 'GC1000 series')]) {
    lead.RecordTypeId = 'Generator'; // Replace 'Generator' with the ID of your Generator lead record type
    leadsToUpdate.add(lead);
}
update leadsToUpdate;
```

This code selects all leads that do not have a record type and have either `GC5000 series`, `GC3000 series`, or `GC1000 series` selected in their `Product_Interest__c` field. It then sets the `RecordTypeId` field to the ID of the `Generator` lead record type and adds those leads to a list to be updated. Finally, it updates the leads in the list.

You can execute this code snippet in the Salesforce Developer Console by selecting "Execute Anonymous" from the "Debug" menu, pasting the code snippet into the window that appears, and clicking "Execute".

[Prompt 5]

Please update your code to refer to Product_Interest__c as ProductInterest__c instead and also exclude any converted leads from updating.

[Response]

Sure! Here's the updated code that refers to `ProductInterest__c` instead of `Product_Interest__c` and excludes converted leads:

```
List<Lead> leadsToUpdate = new List<Lead>();
for (Lead lead : [SELECT Id, RecordTypeId, ProductInterest__c FROM Lead WHERE RecordTypeId = null AND IsConverted = false AND ProductInterest__c IN ('GC5000 series', 'GC3000 series', 'GC1000 series')]) {
    lead.RecordTypeId = 'Generator'; // Replace 'Generator' with the ID of your Generator lead record type
    leadsToUpdate.add(lead);
}
update leadsToUpdate;
```

This code selects all leads that do not have a record type, have not been converted, and have either `GC5000 series`, `GC3000 series`, or `GC1000 series` selected in their `ProductInterest__c` field. It then sets the `RecordTypeId` field to the ID of the `Generator` lead record type and adds those leads to a list to be updated. Finally, it updates the leads in the list.

You can execute this code snippet in the Salesforce Developer Console by selecting "Execute Anonymous" from the "Debug" menu, pasting the code snippet into the window that appears, and clicking "Execute".

[Prompt 6]

Give me execute anonymous code to update all of my open leads (not converted leads) that do not have a Record Type to be updated to the Both record type.

[Response]

Certainly! Here's an example code snippet that you can use to update all open leads that do not have a record type to be updated to the 'Both' record type:

```
List<Lead> leadsToUpdate = new List<Lead>();
        for (Lead lead : [SELECT Id, RecordTypeId FROM Lead WHERE RecordTypeId = null
AND IsConverted = false AND Status NOT IN ('Closed - Converted', 'Closed - Not Converted')])
{
        lead.RecordTypeId = 'Both';   // Replace 'Both' with the ID of your 'Both' lead record type
        leadsToUpdate.add(lead);
}
update leadsToUpdate;
```

This code selects all open leads that do not have a record type, have not been converted, and have a `Status` field that is not equal to `''Closed - Converted''` and `''Closed - Not Converted''`. It then sets the `RecordTypeId` field to the ID of the 'Both' lead record type and adds those leads to a list to be updated. Finally, it updates the leads in the list.

You can execute this code snippet in the Salesforce Developer Console by selecting "Execute Anonymous" from the "Debug" menu, pasting the code snippet into the window that appears, and clicking "Execute".

Avoiding Multiselect Picklists for Multiple Product Interest Leads

[Prompt]

You are a Salesforce expert and consultant. I have a ProductInterest__c picklist field on the Lead object that captures a lead's interest in either one of our generators, one of our power supplies, or potentially both a generator and a power supply. Should I change this to a multi-select picklist? Or would it be preferable to instead create a second picklist field so I could capture their generator product interest in one picklist and their power supply product interest in the other? I have heard negative things about multi-select picklist fields and have been informed I should avoid their use. Can you provide best practices on how to solution this scenario, along with why a multi-select picklist should be avoided.

[Response]

In this scenario, I would recommend creating two separate picklist fields on the Lead object, one for the generator product interest and the other for the power supply product interest. This approach will provide a clear and distinct separation of the lead's interests and will allow for better data analysis and reporting.

While multi-select picklist fields can be useful in certain scenarios, they are generally not recommended for several reasons:

Limited functionality: Multi-select picklist fields have limited functionality and do not provide the same level of control as separate picklist fields. For example, you cannot create dependent picklists or validation rules based on the selected values in a multi-select picklist field.

Difficulty in data analysis: It can be difficult to analyze data in multi-select picklist fields since the values are stored as a single string with semicolons separating each value. This can make it challenging to report on or filter data based on specific values.

Potential data quality issues: Multi-select picklist fields can also lead to data quality issues since users can select multiple values, which can result in inconsistent or inaccurate data.

Therefore, it is generally recommended to use separate picklist fields instead of multi-select picklist fields whenever possible. By creating two separate picklist fields for generator and power supply product interests, you can ensure better data quality, more accurate reporting, and greater control over the data.

Lead Scoring Options

[Prompt]

How can I set up Lead Scoring in Salesforce?

[Response]

To set up lead scoring in Salesforce, you will need to follow these steps:

Define your lead scoring model: Determine the criteria and values that will be used to score your leads. This can include factors such as job title, company size, product interest, website activity, and more. Assign point values to each criterion based on its level of importance.

Create a custom formula field on the Lead object: This field will calculate the lead score based on the criteria you defined in step 1. You can use the formula builder to create a formula that adds up the point values for each criterion.

Set up lead assignment rules: Based on the lead score, you can set up lead assignment rules to automatically assign leads to sales reps or teams. For example, you might assign leads with a score of 50 or higher to your top-performing sales reps.

Monitor and adjust the scoring model: As you collect data and evaluate the effectiveness of your lead scoring model, you may need to make adjustments to the criteria and point values. Keep an eye on your lead conversion rates and adjust your scoring model accordingly.

Salesforce provides a native lead scoring feature called Einstein Lead Scoring, which uses artificial intelligence and machine learning to automatically score leads based on their behavior and demographics. You can also use third-party lead scoring tools that integrate with Salesforce, such as LeanData, Infer, or Lattice Engines.

Regardless of the approach you take, lead scoring can help you prioritize your sales efforts and focus on the leads that are most likely to convert into customers.

[Prompt 2]

Please give me a sample formula field that would be a good example of a Lead Scoring formula for Salesforce.

[Response]

```
IF(
  OR(ISBLANK(Company), ISBLANK(Phone)),
  0,
  IF(
    OR(
      ISPICKVAL(Industry, "Education"),
      ISPICKVAL(Industry, "Nonprofit")
    ),
    10,
    IF(
      AND(
        ISPICKVAL(Industry, "Healthcare"),
        CONTAINS(Title, "Director")
      ),
      9,
      IF(
        AND(
          ISPICKVAL(Industry, "Finance"),
          CONTAINS(Title, "VP")
        ),
        8,
        IF(
          AND(
            ISPICKVAL(Industry, "Technology"),
            CONTAINS(Title, "Manager")
```

```
        ),
        7,
        IF(
          CONTAINS(Company, "Acme"),
          5,
          IF(
            AND(
              ISPICKVAL(Industry, "Manufacturing"),
              CONTAINS(Phone, "555-")
            ),
            3,
            1
          )
        )
      )
    )
  )
)
```

Lead Auto Response Rules

[Prompt]

Update this code to add an additional field for Annual Revenue, which is a Standard Field on the Lead object.

[Prompt 2]

Reintroduce the styling like before, when you used Bootstrap and place Annual Revenue as the last field for the user to enter instead of the first.

After many failed attempts from ChatGPT, I went in and manually added the Annual Revenue field to an old, properly styled version of my form. Here is that updated code:

```
<!-- Add this meta tag to your page head -->
<meta http-equiv="Content-type" content="text/html; charset=UTF-8">

<!-- Add the Bootstrap CSS and JS files to your page head -->
<link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.0/css/bootstrap.min.css">
```

```
<script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"></script>
<script
src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.9.3/dist/umd/popper.min.js"></script>
<script src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.0/js/bootstrap.min.js"></script>

<!-- Add this form to your page body -->
<form action="https://webto.salesforce.com/servlet/servlet.WebToLead?encoding=UTF-8"
method="POST" class="needs-validation" novalidate>

  <input type="hidden" name="oid" value=" ">
<!--Note from Mike - Paste your own org ID between the quotes in the line above. You can get
this from the Company Settings screen inside of setup →
  <input type="hidden" name="retURL" value="https://mikewheelermedia.com">
  <input type="hidden" name="lead_source" value="Web">

  <div class="form-group">
    <label for="first_name">First Name</label>
    <input id="first_name" type="text" name="first_name" class="form-control" maxlength="40"
required>
    <div class="invalid-feedback">
      Please enter your first name.
    </div>
  </div>

  <div class="form-group">
    <label for="last_name">Last Name</label>
    <input id="last_name" type="text" name="last_name" class="form-control" maxlength="80"
required>
    <div class="invalid-feedback">
      Please enter your last name.
    </div>
  </div>

  <div class="form-group">
    <label for="email">Email</label>
    <input id="email" type="email" name="email" class="form-control" maxlength="80" required>
    <div class="invalid-feedback">
      Please enter a valid email address.
    </div>
  </div>

  <div class="form-group">
    <label for="company">Company</label>
    <input id="company" type="text" name="company" class="form-control" maxlength="40">
```

```
  </div>

  <div class="form-group">
    <label for="AnnualRevenue">Annual Revenue</label>
    <input id="AnnualRevenue" type="text" name="annualrevenue" class="form-control"
maxlength="18">
  </div>

  <button type="submit" class="btn btn-primary">Submit</button>

</form>

<!-- Add this script to your page body to handle form validation -->
<script>
  // Enable form validation using Bootstrap
  (function() {
    'use strict';

    window.addEventListener('load', function() {
      var form = document.getElementsByClassName('needs-validation')[0];
      form.addEventListener('submit', function(event) {
        if (form.checkValidity() === false) {
          event.preventDefault();
          event.stopPropagation();
        }
        form.classList.add('was-validated');
      }, false);
    }, false);
  })();
</script>
```

Mass Updating Open Cases to Add Record Types

[Prompt]

I have just introduced Record Types on the Case object in my Salesforce org. How can I mass-update all open cases to the Generator record type?

Note from Mike:

You will likely receive several options from ChatGPT, such as using the Data Loader, building a Process via the Process Builder (which is soon to be retired), or writing Batch Apex Code.

You can ask ChatGPT to write the Batch Apex Code, but this requires saving a file and then calling that batch apex class file from the Execute Anonymous window.

I prompted the following at this point:

[Prompt 2]

Rather than creating a batch apex class and then executing anonymous, can we combine all of the code into an execute anonymous code block so I don't have to create a batch apex class? if so, update the code accordingly.

EXECUTE ANONYMOUS CODE

You can run this code from the Developer Console in the Execute Anonymous Window (accessible from the Debug window). And remember that you can always prompt ChatGPT to add comments to the code or explain it to you line by line. What is especially helpful with the code below is that the Select statement is querying for the DeveloperName for the Record Type, meaning the Name of the Record Type instead of the ID. Since ID's would vary by org, querying by name is a better approach vs. hard-coding in an ID.

```
Id generatorRecordTypeId = [SELECT Id FROM RecordType WHERE SObjectType = 'Case' AND DeveloperName = 'Generator' LIMIT 1].Id;

List<Case> casesToUpdate = [SELECT Id FROM Case WHERE Status NOT IN ('Closed', 'Your Closed Status')];

for (Case c : casesToUpdate) {
    c.RecordTypeId = generatorRecordTypeId;
}

update casesToUpdate;
```

Keep in mind that this approach works well for a smaller number of records. However, if you're working with a large number of records, you may hit the governor limits. In that case, using a batch Apex class is recommended.

To run this code as an Execute Anonymous script:

In the Developer Console, click on "Debug" from the top menu.
Select "Open Execute Anonymous Window" or press CTRL+E.
Paste the Execute Anonymous code block in the window.
Click on the "Execute" button to run the script.

Using ChatGPT to Style Our Web-to-Case Form

[Prompt]

Update this web-to-case form from Salesforce to have a more modern and responsive style, using the Bootstrap framework.

[Prompt 2]

Please provide all of the code needed for your steps 1-3 into one single code box and give me the html file for this in its entirety.

[Prompt 3]

With that updated code, you are not styling the input fields of the form. Please update the code to style the form fields using bootstrap.

[Response]

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Web-to-Case Form</title>

    <!-- Bootstrap CSS -->
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">

    <!-- jQuery library -->
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
```

```
        <!-- Bootstrap JS -->
        <script
src="https://maxcdn.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></script>

</head>
<body>
        <div class="container">
                <h2>Web-to-Case Form</h2>
                <form
action="https://webto.salesforce.com/servlet/servlet.WebToCase?encoding=UTF-8"
method="POST">
                        <input type="hidden" name="orgid" value="">
                        <input type="hidden" name="retURL"
value="https://courses.mikewheelermedia.com">

                        <div class="form-group">
                                <label for="name">Contact Name</label>
                                <input class="form-control" id="name" maxlength="80"
name="name" size="20" type="text" />
                        </div>

                        <div class="form-group">
                                <label for="email">Email</label>
                                <input class="form-control" id="email" maxlength="80"
name="email" size="20" type="text" />
                        </div>

                        <div class="form-group">
                                <label for="phone">Phone</label>
                                <input class="form-control" id="phone" maxlength="40"
name="phone" size="20" type="text" />
                        </div>

                        <div class="form-group">
                                <label for="subject">Subject</label>
                                <input class="form-control" id="subject" maxlength="80"
name="subject" size="20" type="text" />
                        </div>

                        <div class="form-group">
                                <label for="description">Description</label>
                                <textarea class="form-control" name="description"></textarea>
                        </div>
```

```
                    <div class="form-group">
                        <label for="recordType">Case Record Type</label>
                        <select class="form-control" id="recordType" name="recordType">
                            <option value="">--None--</option>
                            <option value=" ">Generator</option>
                            <option value=" ">Power Supply</option>
                        </select>
                    </div>

                    <input class="btn btn-primary" type="submit" name="submit"
value="Submit">
                </form>
            </div>
</body>
</html>
```

Data vs. Metadata

[Prompt]

Explain to me the difference between data vs. metadata as it relates to Salesforce.

Mass Deleting Acquired Accounts

[Prompt]

Please provide me with batch apex code for Salesforce so that I can execute it anonymously via the Developer Console to mass delete all records that belong to this custom object - AcquiredAccount__c.

[Prompt 2]

Combine the code so that I don't have to write a class. I want to execute anonymous

[Response]

Database.delete([SELECT Id FROM AcquiredAccount__c]);

Mass Deleting Future Tasks Greater than One Year From Now

[Follow Up Prompt from Previous Chat]

Please do the same for Tasks, giving me the code to mass delete via the Execute Anonymous window all Tasks in my Salesforce instance that have a due date greater than 1 year from today. I do not want to delete tasks that have a due date in the past or are due in the next year.

[Prompt 2]

I got this error message:

Line: 2, Column: 28
SELECT Id FROM Task WHERE DueDate > :oneYearAgo ^ ERROR at Row:1:Column:27 No such column 'DueDate' on entity 'Task'. If you are attempting to use a custom field, be sure to append the '__c' after the custom field name. Please reference your WSDL or the describe call for the appropriate names.

The api name for DueDate should be ActivityDate

[NOTE FROM MIKE - THIS DELETED ALL TASKS - I AM EXCLUDING RESPONSE AND WILL GIVE CORRECTED CODE IN NEXT LESSON!]

[Response]

Apologies for the mistake. You are correct that the API name for the `Due Date` field on the `Task` object is actually `ActivityDate`. Here's the corrected code:

```
*Date oneYearAgo = Date.today().addYears(-1);*
*List<Task> tasksToDelete = [SELECT Id FROM Task WHERE ActivityDate > :oneYearAgo];*
*Database.delete(tasksToDelete);*
```

To execute this code anonymously via the Developer Console, follow these steps:
1. Open the Developer Console in your Salesforce org.
2. Click on the "Debug" menu, and select "Open Execute Anonymous Window."
3. In the "Execute Anonymous" window, paste the following code:
```
```

```
Date oneYearAgo = Date.today().addYears(-1);
List<Task> tasksToDelete = [SELECT Id FROM Task WHERE ActivityDate > :oneYearAgo];
Database.delete(tasksToDelete);
```

4. Click the "Execute" button to run the code.

This will delete all tasks that have an `Activity Date` greater than 1 year from today. Tasks that have an `Activity Date` in the past or are due in the next year will not be deleted. Note that depending on the number of tasks you have, this process may take some time to complete. Also, keep in mind that this is a permanent delete and cannot be undone, so make sure you have a backup of your data before running this code.

Restoring Records from the Recycle Bin

[Prompt]

This deleted all of my tasks, including those with a due date in the past and within the next year. This should not be oneyearago but oneyearfromtoday. Please update the code and shouldn't the addYears(-1) be (+1) instead?

```
Date oneYearFromToday = Date.today().addYears(1);
List<Task> tasksToDelete = [SELECT Id FROM Task WHERE ActivityDate <
:oneYearFromToday];
Database.delete(tasksToDelete);
```

Using ChatGPT to Generate Sample Data

[Prompt]

Give me sample data in a table format to include fake biographical and demographic information for 25 people. These column headers need to be First Name, Last Name, Street Address, City, State, Zip and Phone.

Using ChatGPT to Create a Python Fake Data Generator

[Follow-Up Prompt from Previous Lesson]

I want a Python program that can generate this same type of fake data for me. Also include a column for fake email addresses for each row of data. I would also like this program to ask the user how many rows they would like generated, up to 1000.

[Response]

```
from faker import Faker

# prompt user for number of rows to generate
num_rows = int(input("How many rows of data would you like to generate (up to 1000)? "))
if num_rows > 1000:
    print("Sorry, the maximum number of rows is 1000.")
    num_rows = 1000

# create faker object
fake = Faker()

# generate fake data
data = []
for i in range(num_rows):
    first_name = fake.first_name()
    last_name = fake.last_name()
    street_address = fake.street_address()
    city = fake.city()
    state = fake.state_abbr()
    zip_code = fake.zipcode()
    phone_number = fake.phone_number()
    email = fake.email()
    data.append([first_name, last_name, street_address, city, state, zip_code, phone_number,
email])

# print data in table format
headers = ["First Name", "Last Name", "Street Address", "City", "State", "Zip", "Phone", "Email"]
print("\t".join(headers))
for row in data:
    print("\t".join(row))
```

[Prompt 2]

Please update this code so that the data which is generated is comma separated.

[Response - Writes a .csv file]

```
import csv
from faker import Faker

# prompt user for number of rows to generate
num_rows = int(input("How many rows of data would you like to generate (up to 1000)? "))
if num_rows > 1000:
    print("Sorry, the maximum number of rows is 1000.")
    num_rows = 1000

# create faker object
fake = Faker()

# generate fake data
data = []
for i in range(num_rows):
    first_name = fake.first_name()
    last_name = fake.last_name()
    street_address = fake.street_address()
    city = fake.city()
    state = fake.state_abbr()
    zip_code = fake.zipcode()
    phone_number = fake.phone_number()
    email = fake.email()
    row = [first_name, last_name, street_address, city, state, zip_code, phone_number, email]
    data.append(row)

# write data to CSV file
headers = ["First Name", "Last Name", "Street Address", "City", "State", "Zip", "Phone", "Email"]
with open('fake_data.csv', 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(headers)
    writer.writerows(data)
```

Creating a Batch Apex Class to Mass Generate Contact Records

[Prompt]

Write batch apex to create 250 contact records for a Salesforce instance.

[Response]

Sure, here is an example of batch apex code that creates 250 contact records in Salesforce:

```
global class CreateContactsBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        String query = 'SELECT Id FROM Account LIMIT 1';
        return Database.getQueryLocator(query);
    }

    global void execute(Database.BatchableContext bc, List<sObject> scope) {
        List<Contact> contactsToInsert = new List<Contact>();
        for (Integer i = 1; i <= 250; i++) {
            Contact newContact = new Contact(
                FirstName = 'Contact',
                LastName = 'Record ' + i,
                Email = 'contact' + i + '@example.com',
                AccountId = ((Account)scope[0]).Id
            );
            contactsToInsert.add(newContact);
        }
        insert contactsToInsert;
    }

    global void finish(Database.BatchableContext bc) {
        // Optional finish method
    }
}
```

This batch apex class uses the `Database.Batchable` interface and implements three methods: `start`, `execute`, and `finish`.

The `start` method returns a `Database.QueryLocator` that retrieves the `Id` of one account record from the Salesforce database. We only need one account record for this example, but you can modify the query to retrieve more accounts if necessary.

The `execute` method generates 250 contact records and adds them to a list of contacts to be inserted. The first contact will be named "Contact Record 1" and the email address will be "contact1@example.com". The last contact will be named "Contact Record 250" and the email address will be "contact250@example.com". The account ID for each contact is set to the ID of the account record retrieved in the `start` method.

The `insert` statement at the end of the `execute` method inserts all of the contact records in the `contactsToInsert` list.

The `finish` method is optional and is not used in this example.

To execute this batch apex class, you can use the following anonymous apex code:

```
Database.executeBatch(new CreateContactsBatch(), 10);
```

This will start the batch job and process 10 records at a time. You can adjust the batch size to fit the needs of your Salesforce instance.

[Prompt 2]

Please also provide me with the apex test class for CreateContactsBatch

[Response]

Sure, here is an example of an Apex test class for the `CreateContactsBatch` class that we created earlier:

```
@isTest
private class CreateContactsBatchTest {

    @isTest
    static void testCreateContactsBatch() {
        // Create a test account record
        Account testAccount = new Account(Name = 'Test Account');
```

```
        insert testAccount;

        // Start the batch job
        Test.startTest();
        CreateContactsBatch batchJob = new CreateContactsBatch();
        Database.executeBatch(batchJob);
        Test.stopTest();

        // Verify that the batch job created 250 contact records
        List<Contact> contacts = [
            SELECT Id, FirstName, LastName, Email, AccountId
            FROM Contact
            WHERE AccountId = :testAccount.Id
        ];
        System.assertEquals(250, contacts.size());
        for (Integer i = 0; i < 250; i++) {
            System.assertEquals('Contact', contacts[i].FirstName);
            System.assertEquals('Record ' + (i+1), contacts[i].LastName);
            System.assertEquals('contact' + (i+1) + '@example.com', contacts[i].Email);
            System.assertEquals(testAccount.Id, contacts[i].AccountId);
        }
    }
}
```

This Apex test class creates a test account record and inserts it into the Salesforce database. Then it starts the batch job by creating an instance of the `CreateContactsBatch` class and calling `Database.executeBatch()` method. The `Test.startTest()` and `Test.stopTest()` methods are used to ensure that the batch job runs in a separate execution context and that any asynchronous processing triggered by the batch job is completed before the test method finishes.

After the batch job has completed, the test class verifies that 250 contact records were created for the test account. It checks that each contact record has the expected first name, last name, email address, and account ID.

Note that this test class assumes that the `CreateContactsBatch` class has been deployed to the Salesforce instance and is available for testing. If you have not deployed the batch class yet, you will need to do so before running this test class.

Using ChatGPT to Generate Validation Rules

[Prompt]

Give me a list of 10 sample Salesforce validation rules that would be considered common use cases across a wide variety of Salesforce objects. Provide these in a 3 column table with the formula for the validation rule in the left column, a title for the validation rule in the middle column, and the description of the rule in the right column.

Formula for Validation Rule	Title of Validation Rule	Description of Rule
`ISBLANK(Email)`	Required Email	Validates that an email address is entered in the email field
`ISPICKVAL(LeadSource, "")`	Required Lead Source	Validates that a lead source is selected for a lead
`ISBLANK(Phone)`	Required Phone	Validates that a phone number is entered in the phone field
`LEN(Description) > 255`	Description Length	Validates that the description field is less than or equal to 255 characters
`NOT(ISPICKVAL(Status, "Closed Won"))`	Won Opportunities	Validates that an opportunity cannot be closed if it is marked as "Closed Won"
`NOT(ISBLANK(Credit_Card_Number__c))`	Credit Card Required	Validates that a credit card number is entered in the credit card field
`NOT(AND(ISBLANK(Phone), ISBLANK(Email)))`	Contact Information Required	Validates that either a phone number or email address is entered
`AND(ISPICKVAL(StageName, "Closed Won"), ISBLANK(CloseDate))`	Closed Won Date Required	Validates that a close date is entered when an opportunity is marked as "Closed Won"
`ISPICKVAL(Status, "Closed")`	Closed Cases	Validates that a case cannot be edited if it is marked as "Closed"
`AND(ISNEW(), ISBLANK(Company))`	Company Required	Validates that a company is entered when creating a new lead

[Prompt 2]

Please give me 5 validation rule formula examples in this same table format to verify that the user has entered a the following in their respective valid formats:

Vehicle Identification Number

Social Security Number
Phone Number
Email Address
Zip Code

[Response]

Formula for Validation Rule	Title of Validation Rule	Description of Rule
`NOT(REGEX(TEXT(Vehicle_Identification_Number__c), "[A-HJ-NPR-Z0-9]{17}"))`	Invalid VIN	Validates that a Vehicle Identification Number (VIN) is entered in a valid format
`NOT(REGEX(TEXT(Social_Security_Number__c), "^(?!000)(?!666)[0-9]{3}(?!00)[0-9]{2}(?!0000)[0-9]{4}$"))`	Invalid SSN	Validates that a Social Security Number (SSN) is entered in a valid format
`NOT(REGEX(TEXT(Phone), "^\\(?([0-9]{3})\\)?[-. ]?([0-9]{3})[-. ]?([0-9]{4})$"))`	Invalid Phone	Validates that a phone number is entered in a valid format
`NOT(REGEX(TEXT(Email), "^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$"))`	Invalid Email	Validates that an email address is entered in a valid format
`NOT(REGEX(TEXT(Zip_Code__c), "^\\d{5}(?:[-\\s]\\d{4})?$"))`	Invalid Zip Code	Validates that a Zip Code is entered in a valid format

Creating a Joined Report

[Prompt]

Give me several scenarios where a joined report would be called for inside of Salesforce

[Response]

A joined report is a type of report in Salesforce that allows you to combine data from multiple report types, giving you a more comprehensive view of your data. Here are several scenarios where a joined report would be useful:

1. When you need to combine data from different objects: Let's say you want to analyze your sales data by region and by product. You have a report for your sales by region and another report for your sales by product, but you need to see the data together to get a complete picture. By creating a joined report, you can combine the data from both reports to see the sales by region and by product in one place.

2. When you need to compare data across different time periods: If you want to see how your sales have changed over time, you might create two reports: one for this year's sales and one for last year's sales. With a joined report, you can easily compare the two reports side-by-side and see how your sales have changed year-over-year.

3. When you need to analyze data at different levels of detail: Let's say you have a report for your sales by account and another report for your sales by product. By creating a joined report, you can drill down into the data at different levels of detail, such as seeing the sales by account for each product or the sales by product for each account.

4. When you need to combine data from different report types: If you have multiple report types with related data, such as opportunities and contacts, you can create a joined report to see how the two types of data are related. For example, you can see how many opportunities each contact has generated or how many contacts are associated with each opportunity.

5. When you need to create a more complex report: Sometimes a single report isn't enough to answer your business questions. By creating a joined report, you can combine multiple reports and data sources to create a more complex report that gives you a more comprehensive view of your data.

Helpful ChatGPT Prompts for Reports and Dashboards

[Prompt]

What are Salesforce Dynamic Dashboards?

[Response]

Salesforce dynamic dashboards are a type of dashboard that allows you to create highly customizable, real-time visual representations of your Salesforce data. These dashboards can be customized based on the user viewing them, with the data shown changing dynamically depending on the user's role or permissions.

With dynamic dashboards, you can create a customized view of data that is specific to each user, team, or department in your organization. These dashboards can be set up to automatically update in real-time, so that users always have access to the most current data.

Dynamic dashboards also allow you to drill down into specific data points, filter and sort data, and view data in various formats such as charts, tables, and graphs. This makes it easy to quickly identify trends, patterns, and insights, and make data-driven decisions.

Overall, dynamic dashboards are a powerful tool for businesses to gain insights into their data and drive better decision-making across the organization.

[Prompt 2]

What are the differences between Summary Formulas and Row-Level Formulas in Reports for Salesforce?

[Response]

In Salesforce, Summary Formulas and Row-Level Formulas are used in reports to perform calculations and display data. The main difference between the two is the level at which the calculation is performed.

Summary Formulas are used to perform calculations at the summary level, which means they operate on a group of records and return a single result. They are used to aggregate data such as totals, averages, and maximum or minimum values. Summary formulas are placed in the "Report Summary" section of a report and are based on the grouping specified in the report.

On the other hand, Row-Level Formulas are used to perform calculations at the individual record level, which means they operate on each record in the report and return a result for each row. They are used to perform calculations on individual fields and can be used to display additional information about each record. Row-level formulas are placed in the "Fields" section of a report and can be based on any field in the report.

In summary, while Summary Formulas are used to perform calculations at the summary level and return a single result, Row-Level Formulas are used to perform calculations at the individual record level and return a result for each row.

[Prompt 3]

How do you add a Bucket Column to a Report in Salesforce and provide me with a scenario where this would prove helpful.

[Response]

To add a Bucket Column to a report in Salesforce, follow these steps:

1. Open the report you want to modify.
2. Click on the "Add" button in the "Fields" pane.

3. Select "Bucket Field" from the dropdown menu.
4. Choose the field you want to bucket on from the "Field to bucket" dropdown.
5. Enter the bucket ranges you want to use in the "Bucket Field Settings" section.
6. Click "Save" to add the bucket column to your report.

A scenario where a bucket column would prove helpful is when you want to group data into ranges or categories. For example, if you have a report on sales data and you want to see how many sales were made in different price ranges, you could use a bucket column to group sales into categories like "Less than $50", "Between $50 and $100", "Between $100 and $200", and "Over $200". This would give you a better understanding of where your sales are concentrated and help you make more informed decisions about pricing and marketing strategies.